



中国大学生服务外包
创新创业大赛



智衣先知

未名湖

详细技术方案

Detailed technical documentation

第十四届中国大学生创新创业大赛（A16 赛道）

目录

1. 项目概述	4
2. 算法调研	5
3. 赛题技术指标达成情况	6
4. 项目创新点	7
4.1 算法创新点	7
4.1.1 自适应的知识蒸馏	7
4.1.2 无解析器的优势	7
4.1.3 外观流特征的级联放大	8
4.2 技术创新点	8
4.2.1 PyTorch 分布式训练	8
4.2.2 APEX 混合精度训练	8
4.2.3 Img2Col 算法优化加速	8
4.2.4 OpenVINO 推理优化	9
5. 算法模型	9
5.1 基于人体解析的模型设计	9
5.1.1 衣服形变模块(Clothe Warp Module)	9
5.1.2 图像生成模块(Image Generate Module)	11
5.2 基于自适应知识蒸馏的模型设计	12
5.2.1 教师网络(Teacher Network)	13
5.2.2 学生网络(Student Network)	13
6. 模型训练	14
6.1 数据收集与处理	14
6.2 模型训练总体架构设计	15
6.2.1 损失函数的设计和优化	15
6.2.2 超参数的设置	16

6.3 训练流程	16
6.3.1 第一阶段	17
6.3.2 第二阶段	18
6.4 算法分析与测试	19
6.4.1 定量分析	19
6.4.2 定性分析	20
7. 项目工程化落地	21
7.1 模型压缩	22
7.1.1 模型剪枝(Model Pruning)	22
7.1.2 模型量化(Model Quantization)	24
7.1.3 模型压缩结果分析	25
7.2 Correlation Module 加速优化	26
7.2.1 算法调研分析	26
7.2.2 Img2Col 算法优化过程	26
7.3 OpenVINO 推理框架的应用	27
7.3.1 技术选型分析	28
7.3.2 模型转换流程	29
7.3.3 OpenVINO 推理性能优化	29
7.4 项目部署	31
7.4.1 部署准备	31
7.4.2 .EXE 编译部署	32
7.4.3 部署中遇到的问题与挑战	32
7.4.4 本地 Web 部署	33
8. 算法评价	36
8.1 定量比较	36
8.2 定性比较	37
9. 算法边界测试	38

9.1 人物非常态姿势	38
9.2 特殊人群换衣	38
9.3 特殊款式衣服	39
9.4 人物与衣服尺寸不匹配	39
9.5 不同衣服款式	39
10. 项目总结和发展前景	40
参考文献	41
附录	42

1. 项目概述

随着互联网的不断普及和发展,越来越多人选择在网上购买自己的所需物品,虚拟试衣概念正是在这样的背景下兴起的,这种技术能够为消费者提供更真实的购物体验,提升消费者的满意度,使顾客能够快速选择最适合自己的服装款式。近年来,京东、阿里, Amazon 等公司纷纷推出了自己的线上虚拟试衣系统,但目前大多数虚拟试穿方法都存在着各种各样的问题,如上身效果差、换装时间长、应用成本高等,不能够实现大规模普及,即便如此,虚拟试衣技术的市场需求仍然呈现增长的趋势[1]。

本项目设计了一种新颖的虚拟试衣算法模型,它包含两个网络模型,教师网络(Teacher Network)和学生网络(Student Network),与目前大多数虚拟试衣算法不同,该模型通过自适应的知识蒸馏(Adjustable Knowledge Distillation),学生网络受到教师网络的监督(人体姿势外观流特征),从而使学生网络摆脱了对人体姿势的解析,避免因错误的人体姿势解析造成不真实的试穿图像。同时,为了解决无解析器的网络即学生网络生成图像质量不高的问题,我们采用了一种采用“Teacher-Tutor-Student”结构的强监督学习模式,即先使用模特的真实穿着图像利用教师网络生成换衣后的图像,再将换衣后的图像输入到学生网络中,在学生网络中,设置要换的衣服为模特的原始着装,最后使用真实着装图像对学生网络生成的图像进行监督和纠正,提高学生网络的准确率。教师网络生成的图像为Tutor,它可以指导学生网络(Student),而学生网络又可以被真实图像(Teacher)所监督,形成一种非常形象的“Teacher-Tutor-Student”结构。

在模型的训练中和训练后,我们使用了大量的优化方法对模型进行压缩和优化,如量化感知训练(Quantization Aware Training)、网络层剪枝(Network Pruning),知识蒸馏(Knowledge Distillation),Img2Col 算法优化等。为了得到一个推理速度快、轻量化的 EXE 可执行文件,我们使用了 OpenVINO 推理框架,并运用了 OpenVINO 的优化工具对模型进行了进一步优化。最后,我们使用 C++语言进行模型部署,得到了最后的 EXE 可执行文件,并对性能与可移植性进行了测试。在完成企业的基本要求后,我们考虑到与用户交互性不足的情况,将我们的模型部署到了 Web 应用上,以便用户可以便捷地使用我们的产品进行换衣体验。

2. 算法调研

在项目初期的调研分析阶段,我们对目前已有的虚拟试衣算法进行了调研和分析。算法调研后发现目前虚拟试衣可以分为基于三维模型和基于二维模型的方法,但由于前者需要额外的三维测量和更为复杂的运算,目前基于 2D 的算法应用更为广泛,而 2D 的虚拟试衣算法又可以分为基于人体解析(Parser-based)和不基于人体解析(Parser-Free)两种方式,我们将基于解析器的前沿方法(左侧)和不基于解析器的方法(右)进行了比较如图 2.1 所示。

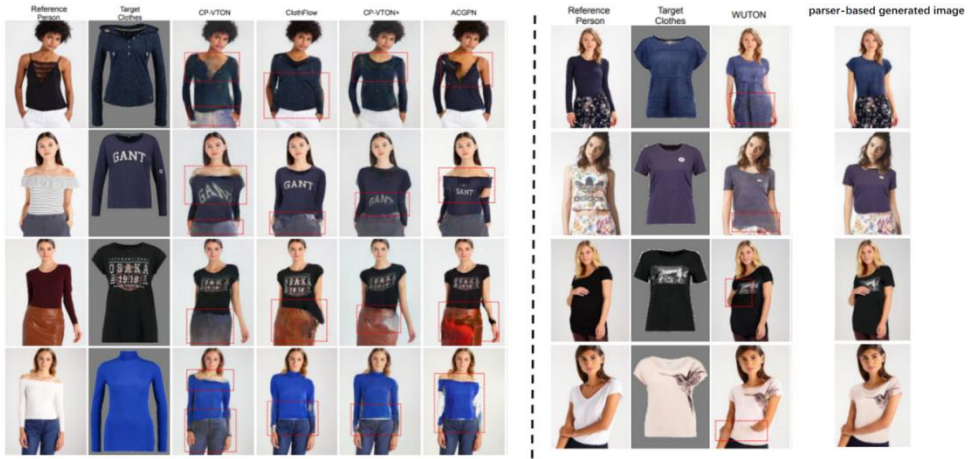


图 2.1 不同的虚拟试衣算法结果比较图

在左侧,我们用红框将不精确的分割区域高亮了出来,这些不精确的区域划分误导了现存的基于解析器的方法,如 CP-VTON[2], ClothFlow[3], CP-VTON+[4]和 ACGPN[5]等方法,导致其最终生成的换衣图像效果不佳,由此我们可以看出基于人体姿势解析的算法模型,其准确性直接受到人体解析准确性的影响。

右侧是一个不基于解析器的方法 WUTON[6],但可以明显地看到, WUTON 方法和基于解析器方法产生的图片效果有一定的差距,前者对人像特殊区域,如胳膊、下装、头发等缺少一定的细节处理,这是因为 WUTON 依然采用传统的知识蒸馏方案,对“教师”网络生成的试穿图像(合成图)去蒸馏一个无需解析器的“学生”网络,该网络不使用分割的特征作为输入,这导致学生直接模仿教师的试穿能力。使用“教师”网络生成的图像作为“Teacher Knowledge”来指导学生网络,它生成的图像质量受到基于解析器的模型也就是“教师”网络的限制。

因此我们在对上述几种算法的不足之处进行分析后,设计了一种基于自适应知识蒸馏的算法模型,使模型摆脱了对人体姿势的解析,避免

因为错误的人体姿势解析造成不真实的试穿图像的问题，不需要复杂的解析器还具备降低模型计算量的优点；同时设计了“Teacher-Tutor-Student”结构，对学生网络生成的图像质量进行监督和纠正，以提高模型的准确性。

3. 赛题技术指标达成情况

万兴科技企业方设置了三项指标来评价最终算法的优劣，分别是 FID，FPS 以及模型大小。

➤ FID（Fréchet Inception Distance）：一种用于评估模型生成图像质量的指标，它的计算公式为

$$FID = ||\mu_r - \mu_g||^2 + Tr(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

其中， μ_r 表示真实图片的特征均值， μ_g 表示生成图片的特征均值， Σ_r 表示真实图片的协方差矩阵， Σ_g 表示生成图片的协方差矩阵， Tr 表示矩阵的迹。

- FPS(Frames Per Second)：每秒显示的帧数，表示图像的流畅程度；
- 模型大小：机器学习或深度学习模型在存储设备上占据的存储空间大小；

以下是我们算法模型技术指标达成情况：

表 3.1 技术指标分布表

技术指标达成情况		
衡量指标	项目实况	赛题要求
FID	8.906	越小越好
	11.503	
FPS	10.79	25
模型大小(MB)	34.95	50

注：赛题测试的 CPU 型号为 i7-8700/10700，FPS 指标已根据主频和 CPU 核心数进行等比例换算，FID 指标在万兴科技提供的数据集(2032 张图片)上测量得到。

4. 项目创新点

4.1 算法创新点

我们对现有开源算法的不足之处进行实验和分析，在算法模型上做出创新，最终将 FID 指标降到了 8.906，已经优于同期大部分论文的算法。

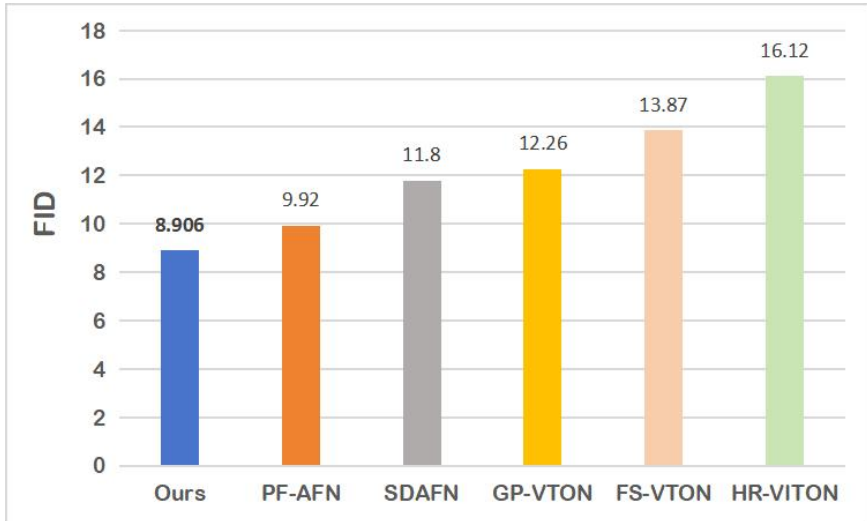


图 4.1.1 FID 指标对比图

也可参看文档第六部分的图 6.4.1 和 6.4.2，我们展示了模型运行的效果示意图。

4.1.1 自适应的知识蒸馏

我们在基于解析器的模型基础上采用了自适应的知识蒸馏技术。通过将待试穿衣服与人体姿势的相关特征蒸馏给学生网络，使得学生网络能够仅获取匹配的相关特征，无需再对人体姿势进行解析。同时，我们对知识蒸馏的方法进行了改进，引入了动态调节因子 ϕ 。这确保只有当教师网络生成高质量的导师图像时，才进行蒸馏。因为当教师网络因解析错误而产生错误扭曲的特征时，蒸馏的模型也会生成失败的结果。这样的改进增加了学生网络对人体姿势的泛化能力和准确度，也是我们获得较好的 FID 值的重要原因之一。

4.1.2 无解析器的优势

基于解析器的模型对人体特征分割的结果及其“敏感”，因为它们严重依赖解析结果来驱动图像生成，当人物的姿势较复杂如遮挡和交叉臂时而难以分割时，这些模型的效果会变得非常差，出现伪影、破损和

不真实的扭曲等。而我们采用的自适应的知识蒸馏技术，将待试穿衣服与人体姿势的相关特征蒸馏给学生网络。这使得学生网络能够摆脱对人体解析的依赖，并自适应地保留或生成身体部位，从而保持身体各处细节。通过这种方式，我们有效解决了基于解析器的模型的局限性，使得虚拟试衣算法在人物出现复杂姿势下仍能表现出更加真实和准确的效果。

由于无需进行复杂的解析过程，极大地简化了模型结构和计算复杂度，使模型更加轻量化和高效，显著提高了推理速度。

4.1.3 外观流特征的级联放大

我们首选采用金字塔网络结构生成流特征[7]，以提取人体姿势特征和衣服图像特征之间的相关性，但发现这种粗粒度的特征表示对衣服的纹理特征、边缘特征以及人体姿势外观流分布的敏感度较低。因此，我们设计了一个特征级联放大网络，该网络将提取的流特征输入放大网络，并逐层进行细粒度卷积再逐级收敛到正常维度大小，以最大程度减少信息损失，使最终生成的试穿图像更加清晰、逼真。

4.2 技术创新点

由于我们选用的模型训练集较为庞大，包含了将近 14000 张人物图片和衣服图片，因此训练的时间也比较长，受到计算资源的限制，我们使用了多种方式加快训练，并引入多项技术不断优化模型的性能。

4.2.1 PyTorch 分布式训练

使用 PyTorch 分布式训练能够充分利用多个设备的计算能力，加速训练过程并提高模型性能，我们使用了 `nn.DistributeDataParallel` 方法实现在多个 GPU 上切分模型和数据，以达到分布式训练的目的。

4.2.2 APEX 混合精度训练

APEX 是来自 NVIDIA 的一个深度学习加速库，在保持模型训练的准确性的前提下，使用较低的数值精度来执行计算操作，从而提高训练速度、提升内存利用效率，在训练中，我们加入了这种 Apex 训练方法，将训练时间缩短了 2.231 倍，并对最终的模型效果进行评估，发现 FID 指标并无明显损失，变化几乎可以忽略。

4.2.3 Img2Col 算法优化加速

Img2Col 是用来高效处理矩阵运算一种算法。该算法不同于传统矩阵进行卷积运算时，需要不断从矩阵中按特定步长截出一定大小的卷积区域。而是充分利用了计算时进行访存的相邻性，将需要计算的特征子

矩阵存放在连续的内存中,便于一次性地将所需要计算的数据直接按照需要的格式取出进行计算,这样便减少了内存访问的次数,从而减小了计算的整体时间,加速模型推理过程。

4.2.4 OpenVINO 推理优化

考虑到模型最终的应用场景:

- Windows 平台
- 不可依靠任何 GPU 资源
- 测试使用的 CPU 处理器型号为 i7-10700/i7-8700

为了充分利用 Intel CPU 的硬件特性,我们选择了 OpenVINO 推理框架来对 PyTorch 模型进行推理和部署。通过 OpenVINO 提供的优化工具,我们对模型进行了优化,包括网络稀疏化(Network Sparsity)、量化感知训练(Quantization Aware Training)等,这些优化技术有效地压缩了模型大小,并提高了模型的推理速度,使得我们的算法在 Windows 平台上有出色的性能表现。

5. 算法模型

5.1 基于人体解析的模型设计

5.1.1 衣服形变模块(Clothe Warp Module)

- 模块整体架构设计

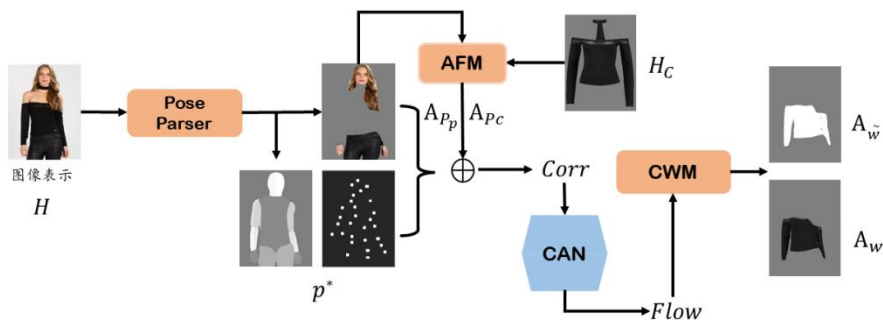


图 5.1.1 衣服形变模块

注: 1、 H 表示模特的真实图像, H_c 表示试穿衣服图像, 我们基于传统解析器的方法从人物图像 H 中解析人体姿势特征表示 p^* ;

2、将人物的解析结果与衣服图像 H_c 经过 AFM(Appearance Flow

Module)提取后得到的流特征 A_{P_p} 、 A_{P_c} , 进行互相关层(Correlation Layer)

卷积运算得到试穿衣服对于人体姿势的特征匹配度表示 $Corr$;

3、为了获取试穿衣服的纹理特征以及边缘部分信息需要对 Correlation Layer 提取出的相关性粗粒度特征进行细化, 因此设计一个级联放大网络(Cascade Amplification Network), 通过将 $Corr$ 先卷积放大可以获得更细致的信息, 再逐步收敛保证能够对网络信息进行细化;

4、CWM(Cloth Warp Module)是一个级联放大后的相关特征 $Flow$ 的网络, 由该网络可以合成映射衣服图像的轮廓特征表示 A_w 和衣服图像的特征表示 $A_{\tilde{w}}$;

➤ 模块网络结构设计

我们设计衣服形变模块主体网络结构分为三个部分, 即衣服外观流特征提取网络、级联放大网络以及形变衣服特征匹配网络, 具体结构如图 5.1.2 所示:

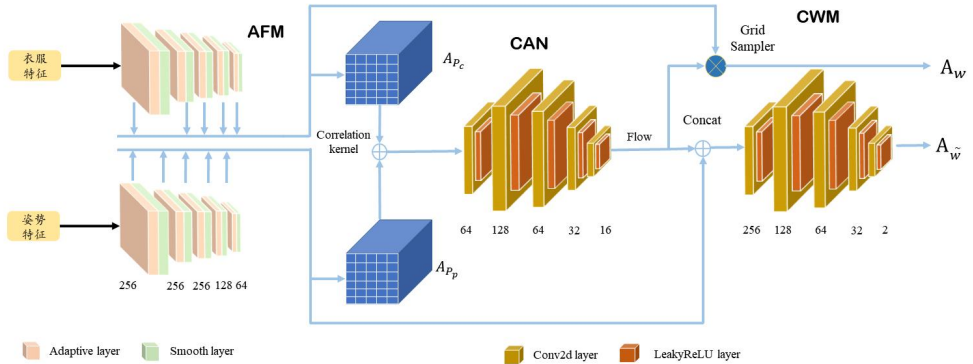


图 5.1.2 衣服形变网络

a. 衣服外观流特征提取网络:

对于人体解析器解析出的人体姿势特征以及衣服特征, 利用特征编码器对其进行编码, 再设计一个由适应层和平滑层构成的五级外观流特征提取层, 对于每一级提取的特征 A_{P_c} 、 A_{P_p} , 汇入到特征总线上形成多维度的特征表示, 虽然这个金字塔结构的网络对特征的提取是粗粒度的 (卷积核逐级扩大), 但是能够利用金字塔不同层级对输入特征进行初步挖掘, 这样能够为后面的细化工作提供有力支持。

b. 级联放大网络:

级联放大网络的设计是由两级卷积层就行特征放大的, 之后对于放大的特征表示进行逐级卷积细化, 通过不断扩展卷积核大小控制卷积特征的逐步收敛, 最终实现对衣服纹理特征的提取和边缘化处理得到细化

后的相关特征表示 $Flow$ 。

c. 形变衣服特征匹配网络：

设计匹配网络一方面要对试穿的衣服进行二维双线性插值以达到形变衣服匹配人体姿势的目的，另一方面人体姿势特征 A_{p_p} 结合相关特征 $Flow$ 以得到匹配的姿势特征即人体待试穿区域表示，其中人体姿势特征匹配也借鉴了级联放大网络的设计结构，对将 $Flow$ 和 A_{p_p} 结合在一起的特征进行放大输出得到 A_w 、 $A_{\tilde{w}}$ 。

5.1.2 图像生成模块(Image Generate Module)

➤ 模块整体架构设计

为了能够较好的综合衣服形变特征和人体姿势特征即进行试衣图像的合成，我们设计了一个基于 Res-Unet 网络的 GM 网络模块，具体结构如图 5.1.3 所示：

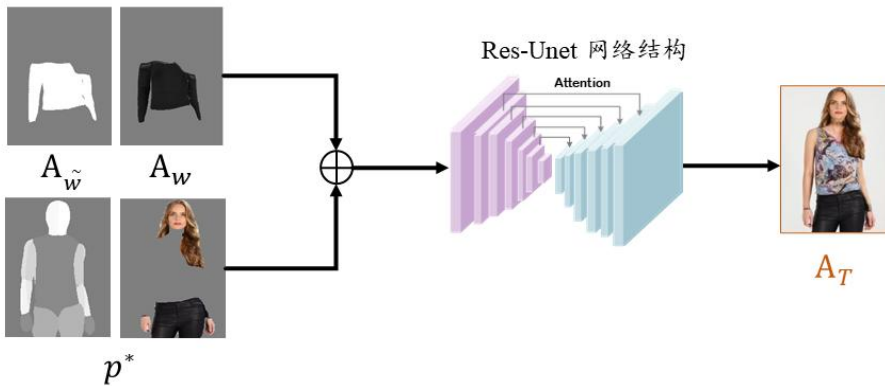


图 5.1.3 图像生成模块

基于姿势解析特征 p^* 和形变特征 A_w 、 $A_{\tilde{w}}$ 的生成模块(IGM)将扭曲的衣服、人体姿态估计和保留的身体区域连接起来作为输入，模块采用 Res-Unet，结合残差连接，可以保留扭曲的衣服的细节并生成逼真的试穿结果。

➤ 模块网络结构设计

我们设计图像生成模块主体网络结构如图 5.1.4 所示，Res-Unet 的网络结构包括一个收缩路径和一个扩张路径。收缩路径遵循卷积网络的典型结构。它包括重复应用两个 3×3 卷积（未填充的卷积），每个卷

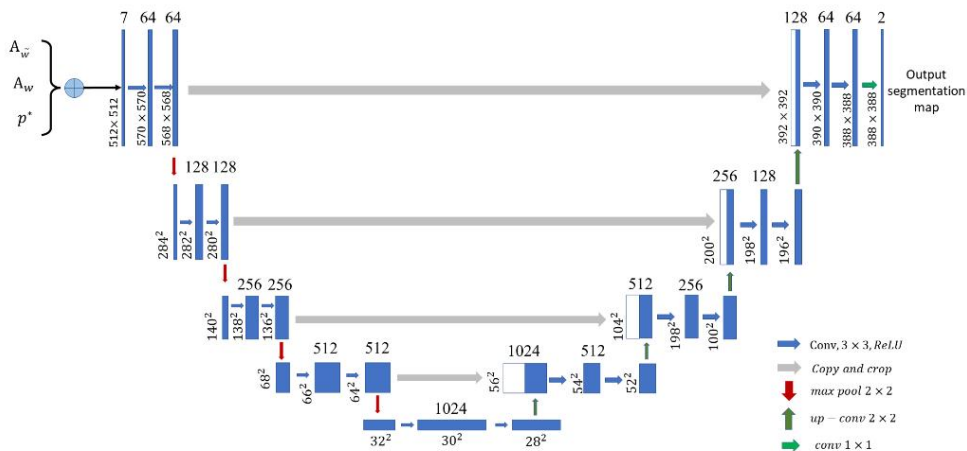


图 5.1.4 图像生成网络

积后都有一个整流线性单元(ReLU)和一个 2×2 的最大池化操作即跨度为 2 的下采样。在每个下采样步骤中，我们将特征通道的数量增加一倍。扩张路径中的每一步都包括对特征图的上采样。

然后进行 2×2 卷积（"上卷积"），将特征通道的数量减半，与收缩路径中相应裁剪的特征图进行串联，并进行两次 3×3 卷积，每次都有一个 ReLU 跟随。由于每次卷积都会损失边界像素，因此裁剪是必要的。在最后一层，一个 1×1 卷积被用来将每个 64 分量的特征向量映射到所需数量的类别上。该网络总共有 23 个卷积层。而 Res-UNet 建立在 UNet 架构的基础之上，结合残差连接，可以保留扭曲的衣服的细节并生成逼真的试穿结果。

5.2 基于自适应知识蒸馏的模型设计

前文所述基于人体解析的模型，经过测试发现虽然模型推理速度以及模型大小均完美符合赛题要求，但是生成的试穿图像质量不高，效果不佳。因此我们采用了自适应的知识蒸馏方法重新设计了两个主干网络即教师网络和学生网络。

我们团队的设计思路是将原始图像利用上述设计的基于解析器的模型进行虚拟试穿，得到试穿的图像后通过设计一个基于教师网络相关性特征的学生网络对其试穿原始的衣服，最后引导学生网络能够摆脱对人体解析器的依赖。

模型总体设计架构图如图 5.2.1 所示：

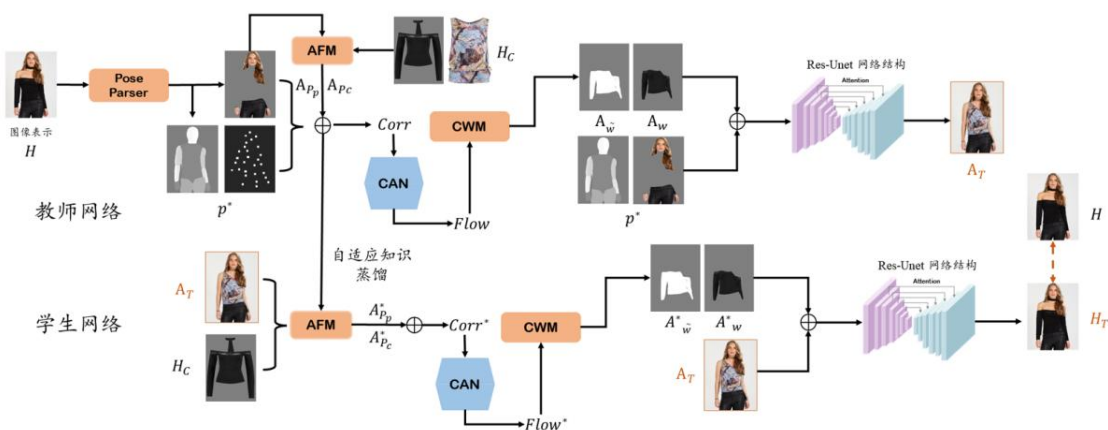


图 5.2.1 教师-导师-学生网络

5.2.1 教师网络(Teacher Network)

教师网络第一部分是外观流模块 AFM(Appearance Flow Module), 该模块的作用是提取衣服图像和人体姿势图像之间的相关特征, 以便对衣服进行形变; 同时为了使衣服能够紧密贴合人的图像并在细节方面保留衣服的特征, 如标志和条纹, 对提取出的 Corr 特征设计了一个级联放大网络, 使得粗粒度的相关特征表示能够转换为细粒度的姿势匹配流特征 Flow。Flow 描述了 H 中的服装与 H_c 的采样位置关系, 由此可对衣服做形变扭曲, 使其能够填充 H 图像中的像素, 也就是变形为穿在 H 图像的人身上的样子。再利用 CWM 模块进行二维双线性插值采样对衣服 H_c 做变形得到 A_w 以及人体的待匹配区域 $A_w^{\sim}$ 。

教师网络第二部分是基于人体解析的图像生成模块 IGM(Image Generation Module), 该模块的作用是利用设计的 Res-Unet 网络结构对经过 CWM 提取得到的 A_w 和 $A_w^{\sim}$ 进行综合生成学生网络所需要的导师图像 A_T 。

5.2.2 学生网络(Student Network)

从图 5.2.1 中可以看出, 待试穿衣服图像特征和人体姿势特征有高度的相关性, 即自相关层卷积计算得到的 Corr, 因此我们考虑将这部分相关特征作为学生网络的指导知识, 直接摆脱对人体姿势解析的依赖, 获取相关性特征, 进而可以对导师图像进行反向虚拟试穿。

➤ 自适应的知识蒸馏方案

我们团队提出了一种自适应的知识蒸馏方案, 在提取衣服和人体姿势相关性特征时引入了动态调节因子 φ , 能够保证找到它们之间的密集对应关系。

教师网络的输入包括人体姿势解析特征、Densepose 估计和人物图

像的姿势估计。相反，学生网络的输入仅为导师图像和衣服图像。因此，在大多数情况下，从教师网络提取的特征通常可以捕捉到更丰富的语义信息，并且估计的外观流更准确，因此可以用于指导学生网络。但是，如果分割结果不准确，教师网络将提供错误的指导，使其语义信息和预测的流失去可靠性。为了解决上述问题，我们引入了一种新颖的自适应知识蒸馏方式，以确保只有准确的表示和预测得以保留。动态调节因子 ϕ 用于决定是否启用蒸馏，通过利用教师图像来评估所生成图像的质量。如果来自基于解析器的教师网络的生成图像 A_T 的质量不如基于解析器的学生网络的生成图像 H_T ，则蒸馏将被禁用；如果来自基于解析器的教师网络的生成图像的质量优于基于解析器的学生网络的生成图像，我们会动态的增大调节因子，并根据优化的程度动态调整自适应知识蒸馏的程度，从而保证蒸馏过程始终朝着更优的方向传递有效知识。

6. 模型训练

6.1 数据收集与处理

我们选用了虚拟试衣技术相关的开源数据集——VITON[12]，VITON-HD[13]和MPV[14]数据集。其中 VITON 是包含 14,221 张图像的训练集和 2,032 张图像的测试集，每个图像对均包含分辨率为 256×192 的正面女性照片和上衣图像。以前的大多数虚拟试穿工作都使用此数据集进行训练和验证。VITON-HD 与 VITON 相同，只是图像分辨率为 512×384 。由于以前没有对这个分辨率的数据集进行过处理，因此生成逼真的试穿结果对于高分辨率的输入非常具有挑战性。MPV 包含 35,687 张人物图像和 13,524 张服装图像，分辨率均为 256×192 ，并拥有一个由 4175 对图像组成的测试集。

我们对收集到的数据进行了数据预处理，数据预处理的大致分为以下几个步骤：

- 1) 去除低质量的图片和标注：在开源数据集上，有些图片和标注的质量较低，会对模型的训练造成影响，因此我们团队使用了人工筛选和自动过滤的方法去除这些低质量的图片和标注；
- 2) 图片和标注的对齐：为了在图像生成任务中实现虚拟试衣，必须将衣服的图像“对齐”到人体图像上，我们使用了 HRNet-W32[8]算法对衣服图像进行分割，对于每个人区域需要估计其对应的 DensePose 和姿态信息，DensePose 用于提取人体表面的 UV 纹理坐标，并将其对应到通用的空间坐标系中，以方便后续的处理。姿态信息包括身体的旋转角度和关节点位置，这里，我们采用的是较为先进的 OpenPose 算法[9]。

- 3) 图片的裁剪和尺寸调整：对于人体区域和其对应的衣服图像，需要将它们缩放到固定的大小以便神经网络的输入，我们选择了 256×192 的图片尺寸，将调整后的人体区域和衣服图像拼接在一起，用于训练和测试。
- 4) 数据增强：采用随机反转、随机旋转、随机裁剪等方法对输入数据进行增强，同时，为了避免过拟合(over-fitting)，还对衣服的纹理进行了随机扰动，以增加训练数据的多样性。这些步骤可以有效提高最终模型的鲁棒性和泛化能力。

6.2 模型训练总体架构设计

6.2.1 损失函数的设计和优化

我们设计了一个综合评估损失函数来优化模型，主要包含以下几个部分：

- 1) 重建损失 (Reconstruction Loss)：重建损失衡量生成的虚拟试衣图像与真实试衣图像之间的差异。在这里，我们使用了 L_{rec} 损失作为重建损失，可以表示为：

$$L_{rec} = ||G(l_s, l_t) - l_g||_1$$

其中： G 表示生成网络， l_s 表示源图像， l_t 表示目标图像， l_g 表示真实试衣图像。

- 2) 感知损失 (Perceptual Loss)：感知损失 L_{perc} 通过比较生成的虚拟试衣图像和真实试衣图像在特定层次上的特征表示来衡量它们之间的差异。这样可以确保生成的图像在高级特征上与真实图像相似。这里我们使用了 VGG-19 网络的某些层次来计算感知损失。
 - 3) 风格损失 (Style Loss)：风格损失 L_{style} 用于确保生成的虚拟试衣图像在风格上与源图像服装相似。这里我们使用了 Gram 矩阵来计算风格损失。
 - 4) 服装区域对齐损失 (Clothing Region Alignment Loss)：这个损失项 L_{align} 用于确保源图像中的服装区域与生成的虚拟试衣图像中的服装区域对齐。这里使用了一个鉴别器网络来衡量源图像和生成图像之间的对齐程度。
 - 5) 平滑损失 (Smoothness Loss)：平滑损失 L_{smooth} 是为了减少生成图像中的噪声和伪影，通过计算生成图像的梯度值的平方和来实现。
- 综合以上五个损失项，可以得到总损失函数 L_{total} ：

$$L_{total} = \lambda_{rec} * L_{rec} + \lambda_{perc} * L_{perc} + \lambda_{style} * L_{style} \\ + \lambda_{align} * L_{align} + \lambda_{smooth} * L_{smooth};$$

其中 λ_{rec} 、 λ_{perc} 、 λ_{style} 、 λ_{align} 和 λ_{smooth} 分别是各损失项的权重。

通过优化这个综合评估损失函数，模型可以生成更高质量的虚拟试衣图像，同时保持与源图像服装的外观特征和风格一致性。

6.2.2 超参数的设置

考虑到不同数据集的训练集和测试集数量不同，以及图像的分辨率也不完全一致，因此对于特定的数据集需要设定不同的超参数并且各参数约定表示如表所示：

表 6.2.1 参数约定表

训练超参数	符号表示
初始学习率	LR
训练轮次数	EP
训练批次	BS
单批次迭代数	IT

具体不同数据集的各参数设置见表 6.2.2 所示：

表 6.2.2 超参数配置表

模型	TCWM				TIGM			
超参数	LR	EP	BS	IT	LR	EP	BS	IT
VITON	0.00005	100	96	50	0.00005	100	64	50
VITON-HD	0.00005	100	96	50	0.00005	100	64	50
MPV	0.00005	200	96	100	0.00005	200	64	100

模型	SCWM				SIGM			
超参数	LR	EP	BS	IT	LR	EP	BS	IT
VITON	0.00003	200	45	100	0.00005	100	32	50
VITON-HD	0.00003	200	45	100	0.00005	100	32	50
MPV	0.00003	200	45	100	0.00005	200	32	100

6.3 训练流程

对设计的基于可微调知识蒸馏的综合网络模型，训练主要分两个阶段，第一阶段是对教师模块的训练，以获得基于解析器进行初步试衣的蒸馏知识；第二个阶段即是在获取到相应试穿的知识后，学生模块借用导师知识图像训练试穿预测模型。

为了提高模型训练效率和轻量化模型结构，我们采用了以下整体训练技术：

➤ Apex 混精训练模型:

APEX 是来自英伟达 (NVIDIA) 的一个深度学习加速库,完美支持 PyTorch 框架,用于改变数据格式来减小模型显存占用的工具。其中最有价值的是 Amp (Automatic Mixed Precision), 将模型的大部分操作都用 Float16 数据类型测试,一些特别操作仍然使用 Float32。实验证明,使用 Float16 作为大部分操作的数据类型,并没有降低参数,在一些实验中,反而由于可以增大 Batch size,带来精度上的提升,以及训练速度上的提升。

➤ Pytorch 分布式训练:

基于 PyTorch 深度学习框架的模型训练,有两种方法可以实现在多个 GPU 上切分模型和数据,即 `nn.DataParallel` 和 `nn.DistributedDataParallel`。由于前者不能使用 Apex 进行混合精度训练,因此我们选择使用 `nn.DistributedDataParallel`。

`DistributedDataParallel` 通过创建多个进程并在多个 GPU 间复制模型,每个 GPU 都由一个进程控制。GPU 可以都在同一个节点上,也可以分布在多个节点上。每个进程都执行相同的任务,并且每个进程都与所有其他进程通信,并且进程或 GPU 之间只传递梯度,以避免通信瓶颈的问题。

6.3.1 第一阶段

教师模块也分为两个网络结构,即在模型设计中提到的衣服形变模块和图像生成模块,模型训练技术选型:

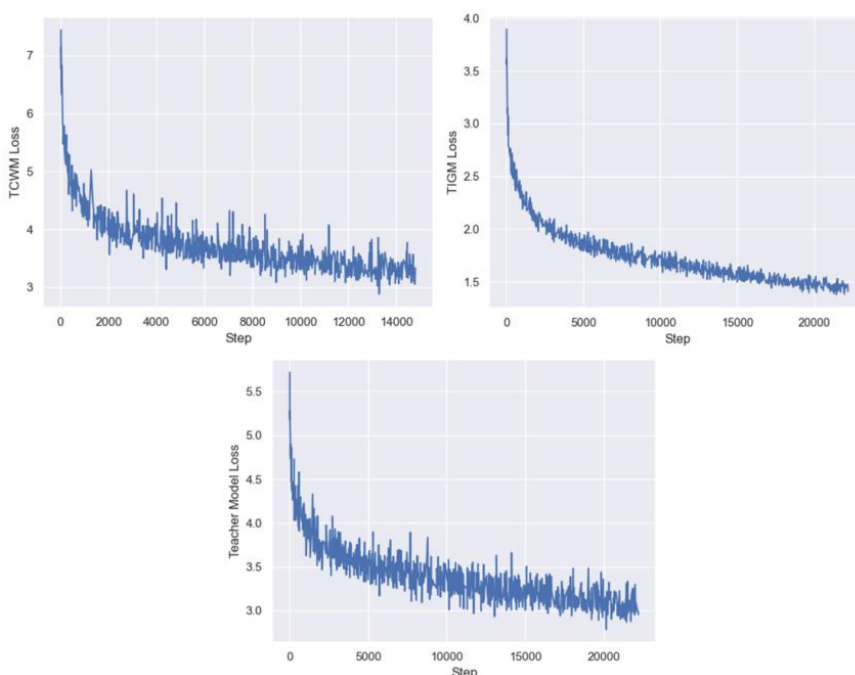
➤ 消融实验技术

对于这两部分网络考虑到模型结构较为复杂训练耗时较长,因此我们采取消融实验思想进行模型的分离训练,即先训练衣服形变模块 A,再加载训练完成的模型 FA(Finished A)然后进行图像生成模块 B 的训练即 FA + B,得到训练完成的模型 FB(Finished B);

具体训练过程中部分定性和总体定量结果展示如下图所示:



(a)



(b)

图 6.3.1 教师网络训练损失变化图

6.3.2 第二阶段

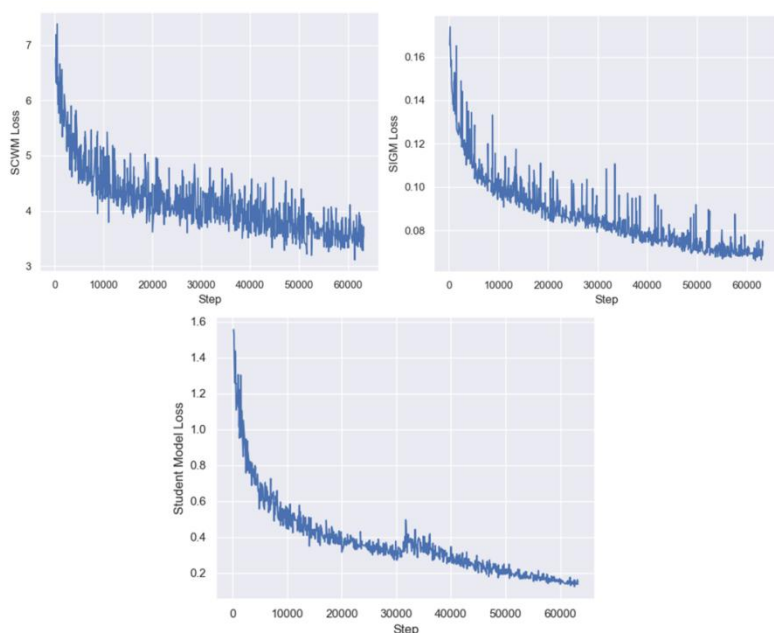
学生模块基于导师图像知识进行相应的特征提取和模型推理，其整体结构与教师模块大体相同，此外为了能够较好的蒸馏教师网络使得学生网络有更好的鲁棒性和泛化能力，对于这一模块也采用消融技术进行训练。

对于第一阶段训练好的模型 FA、FB，我们将其同时加载到学生模块的衣服形变模块，在对 FA、FB 完成推理的基础上得到用于指导学生模块的导师图像 T_H ，进而利用其作为学生模块衣服形变网络的输入。整体训练模式为 FA + FB + C，由此得到学生模块第一个网络的已训练模型 FC；

学生模块图像生成网络的训练：在前面的训练基础上，依据训练模式 FA + FB + FC + D，进行最后网络的训练。同前面训练过程，在 FA + FB + FC 完成相应的模型推理生成初步的特征表示，作为模型 D 的输入，完成整个流程的训练。具体训练过程中部分定性和整体定量结果展示如图 6.3.2 所示。



(a)



(b)

图 6.3.2 学生网络训练损失变化图

6.4 算法分析与测试

6.4.1 定量分析

我们采用 Frechet Inception Distance (FID)作为评估指标,该指标可以捕捉生成的图像与真实图像(即参考人图像)之间的相似度,从根本上来讲,FID 是在衡量两个多元正态分布的距离,FID 值越低表示生成图片与真实图片越接近。其计算公式如下所示:

$$FID = ||\mu_r - \mu_g||^2 + Tr(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

其中, μ_r 表示真实图片的特征均值, μ_g 表示生成图片的特征均值, Σ_r 表示真实图片的协方差矩阵, Σ_g 表示生成图片的协方差矩阵, Tr 表示矩阵的迹。

根据上述计算流程, 我们最终FID测试结果如下:

表 6.4.1 不同数据集上 FID 指标测试结果

数据集	赛题测试集	VITON[12]	VITON-HD[13]	MPV[14]
FID	8.906	9.13	8.99	9.35

为了更加全面地衡量虚拟试衣效果的质量, 我们除了使用了万兴科技要求的 FID 指标, 还使用了结构性相似指数(SSIM)和峰值信噪比(PSNR), 而且还和其他的虚拟试衣算法进行了比较, 具体可见技术文档的 6.2.1 节。实验结果显示, 相较于其他的算法, 我们的模型在多个数据集上实现了更高的准确性和较低的运算成本, 并且在多个方面都具有优越的性能, 在保留原始图像质量的同时, 提高了虚拟试衣的准确性, 为实际应用提供了有力支持。

6.4.2 定性分析

从直观角度衡量我们的模型的运行效果, 表现如下:



图 6.4.1 常见测试样例展示图

除了对基本的短袖、长袖等常见衣物, 我们还实现了连衣裙、背心、外套、卫衣以及人物的背面试衣, 同时还支持人物的特殊姿势试衣,

比如侧身或者复杂手势，具体效果展示如图所示：



图 6.4.2 特殊测试样例展示图

可以看到，我们模型的测试结果无论从定量(FID)还是定性感观上看效果都比较优秀，生成的效果图片更加逼真，有效。

7. 项目工程化落地

对于设计并训练的模型，我们需要进一步将其转换为可部署的应用产品，实现项目的工程化落地。在这个过程中，我们需要考虑多个关键问题，包括算法实际效果、模型大小、推理速度和应用形式等。

为了实现高效合理的部署，我们运用了一系列相关技术，如图所示是本项目开展工程化落地的工程部署总图：

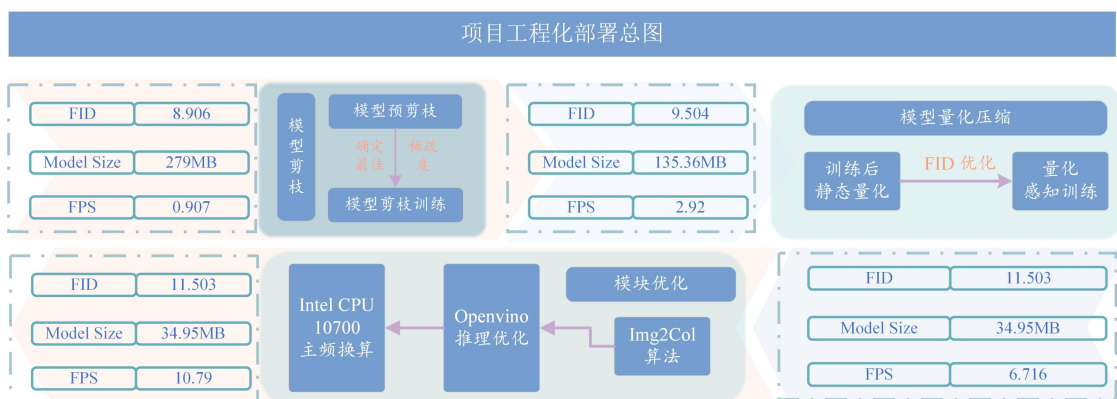


图 7.1 项目工程化部署总图

7.1 模型压缩

基于 Pytorch 模型，我们为了实现模型的轻量化部署和加速推理，进一步对模型进行优化，本节主要介绍模型压缩的技术实现方案，我们分别采用了模型剪枝和模型量化压缩两种方法以提升其性能。

7.1.1 模型剪枝(Model Pruning)

模型剪枝是去除深度学习网络模型中从卷积层到全连接层存在着大量冗余参数的神经元，这些神经元激活值趋近于 0，将这些神经元去掉后可以表现出同样的模型表达能力，因此能够在保持精度损失很小的情况下，轻量化模型从而加快推理速度。目前比较主流的剪枝方法主要有权重剪枝、神经元剪枝、卷积核剪枝、层剪枝。

我们按照不同网络对应不同稀疏度的原则进行适应性剪枝，同时考虑到赛题方对于模型效果极为看重，即模型的 FID 指标越小越好，因此我们在实际开展模型剪枝技术方案时，将此阶段分为模型预剪枝和模型剪枝训练两个部分。模型预剪枝主要是为了通过初步实验探索出模型的剪枝率与 FID 损失的变化关系，寻找一组 FID 损失不大并且模型能够实现压缩的实际剪枝方案；模型剪枝训练主要是为了进一步地提升模型的性能，通过前一部分寻找的最佳剪枝率组合重新定义网络结构进行训练，可以在一定程度上弥补剪枝带来的精度损失。

➤ 模型预剪枝

在该部分，我们探索了不同模型对应于不同的网络稀疏度所得到的网络总运算量、参数量以及 FID 指标，具体模型剪枝方案及数据结果如下表所示：

表 7.1.1 通道剪枝技术方案实验结果记录表

Channel Pruning--Model Metrics Table							
Model	Clothe Warp Module						
Metrics	GFLOPs	Para(M)	SIZE(MB)	Total SIZE(MB)	Compression Ratio	FID	FID Loss
Original Model	6.63	9.37	35.8	112.00	100.00%	8.906	0.00%
Ratio = 0.2 and FineTuning	5.23	7.28	27.6	88.69	79.19%	9.013	1.20%
Ratio = 0.3 and FineTuning	4.40	6.48	24.8	65.73	58.69%	9.113	2.32%
Ratio = 0.4 and FineTuning	3.79	5.61	20.4	40.97	36.58%	9.304	4.47%
Ratio = 0.5 and FineTuning	3.42	4.55	16.8	35.47	31.67%	9.977	12.03%
Model	Image Generation Module						
Metrics	GFLOPs	Para(M)	SIZE(MB)	Total SIZE(MB)	Compression Ratio	FID	FID Loss

Original Model	21.93	43.90	167	167	100.00%	8.906	0.00%
Ratio=0.2 and FineTuning	16.54	35.02	112.3	112.3	67.25%	9.212	3.44%
Ratio=0.25 and FineTuning	15.45	31.93	94.39	94.39	56.52%	9.405	5.60%
Ratio=0.3 and FineTuning	13.90	29.89	80.25	80.25	48.05%	9.679	8.68%
Ratio=0.35 and FineTuning	12.78	27.31	73.49	73.49	44.01%	9.835	10.43%
Ratio=0.4 and FineTuning	11.20	26.12	68.52	68.52	41.03%	10.527	18.20%

➤ 模型剪枝训练

在该部分，通过模型预剪枝确定了最佳的剪枝率组合，利用预剪枝结果重新定义网络结构，以 ICM 模型的各模块为例展示如下表所示：

表 7.1.2 模型剪枝前后通道数对比分析表

Module	Sequence	Sub Blocks	未剪枝通道数	剪枝后通道数	Module	Sequence	Sub Blocks	未剪枝通道数	剪枝后通道数
Image Features	S0	Block 0	64	64	Cond Features	S0	Block 0	64	64
		Block 1	64	32			Block 1	64	32
		Block 2	64	64			Block 2	64	64
		Block 3	64	32			Block 3	64	32
		Block 4	64	64			Block 4	64	64
	S1	Block 0	128	128		S1	Block 0	128	128
		Block 1	128	64			Block 1	128	64
		Block 2	128	128			Block 2	128	128
		Block 3	128	64			Block 3	128	64
		Block 4	128	128			Block 4	128	128
	S2	Block 0	256	256		S2	Block 0	256	256
		Block 1	256	128			Block 1	256	128
		Block 2	256	256			Block 2	256	256
		Block 3	256	128			Block 3	256	128
		Block 4	256	256			Block 4	256	256
Image Features	S3	Block 0	256	256	Cond Features	S3	Block 0	256	256
		Block 1	256	128			Block 1	256	128
		Block 2	256	256			Block 2	256	256
		Block 3	256	128			Block 3	256	128
		Block 4	256	256			Block 4	256	256
	S4	Block 0	256	256		S4	Block 0	256	256
		Block 1	256	128			Block 1	256	128
		Block 2	256	256			Block 2	256	256
		Block 3	256	128			Block 3	256	128
		Block 4	256	256			Block 4	256	256

根据以上数据结果，在 FID 损失可以接受的条件下，最终确定最佳的剪枝率组合为：

表 7.1.3 模型通道剪枝实验数据表

模型	原模型	稀疏度	剪枝后	FID	FPS
CWM	112MB	40%	40.97MB	9.504	2.92
IGM	167MB	25%	94.39MB		

7.1.2 模型量化(Model Quantization)

1) 技术选型

基于 PyTorch 框架，为进一步对模型进行轻量化和推理速度优化，我们分别尝试了训练后量化(Post-Training Static Quantization)和量化感知训练(Quantization Aware Training)两种量化技术方案，并最终采用了量化感知训练作为最终的模型量化方案。量化是一种加速推理的技术，指使用精度较低的数据格式(如 int 8)替换默认的数据格式(float 32)进行计算和内存访问的技术，与浮点实现相比，可以在以下几个重要方面实现性能提升：

- 模型总体尺寸缩小为原来的 $\frac{1}{4}$
- 内存带宽减少为原来的 $\frac{1}{2} \sim \frac{1}{4}$
- 提高推理速度接近 4 倍

2) 技术核心关键难点

在对模型进行量化压缩前，需要指定若干网络层的融合，以便达到最佳性能，我们首先对网络层进行融合，具体方案见下表所示：

表 7.1.4 网络层融合方案表

网络层融合方案	网络层融合后
Conv+BN	ConvBN
Conv+BN+ReLU	ConvBNReLU
BN + ReLU	BNReLU

➤ 训练后量化 (Post Training Quantization, PTQ)

对于训练后量化技术又分为训练后动态量化和训练后静态量化，训练后静态量化通常在内存带宽和计算节省都很重要时使用，因此我们首先选择了训练后静态量化技术方案。训练后静态量化量化了模型的权重和激活值，在可能的情况下，它将激活值融合到前一层中，并且需要用一个具有代表性的数据集进行校准，以确定模型的最佳量化参数。

但量化技术通过减少数据表示位数，会对模型的精度产生一定的损

失，因此我们选取模型的重要指标 FID 来定量分析其精度损失的影响，具体数据结果见下表所示：

表 7.1.5 训练后静态量化 FID 指标

模型优化	CPU-FID	GPU-FID	Model Size
量化前	9.504	9.483	135.36MB
量化 CWM	10.879	10.835	40.97MB → 10.95MB
量化 IGM	12.932	12.848	94.39MB → 25.03MB
量化 CWM、IGM	15.378	15.282	135.36MB → 35.98MB

➤ 量化感知训练(Quantization Aware Training, QAT)

量化感知训练（QAT）模拟了训练过程中量化的影响，量化的缩放因子与零点都会与网络中所有的权重以及激活值绑定，在训练过程中通过模拟量化的手段使得量化的结果不断逼近 FP32 模型，随着训练过程的推进，每组缩放因子与零点也会达到最接近 FP32 模型的程度。在模型转换之后，权重和激活值被量化，激活被尽可能地融合到前一层中。因此量化感知训练可以允许更高的精度。

表 7.1.6 量化感知训练 FID 指标

模型优化	CPU-FID	GPU-FID	Model Size
量化前	9.504	9.483	135.36MB
量化 CWM	9.783	9.701	40.97MB → 10.85MB
量化 IGM	10.382	10.249	94.39MB → 24.10MB
量化 CWM、IGM	11.503	11.379	135.36MB → 34.95MB

7.1.3 模型压缩结果分析

对于模型压缩技术方案，应用模型剪枝和模型量化两种技术后，原始模型的大小非常好地完成了赛题方的要求，我们从理论上和实际上两个方面分别展示选定的模型压缩技术方案的数据结果，如表 7.1.7 和表 7.1.8 所示：

表 7.1.7 模型压缩理论技术方案表

模型	原模型	稀疏度	剪枝后	量化后	总模型大小	FPS
CWM	112MB	40%	67.20MB	16.80MB	48.20MB	5.84~11.68
IGM	167MB	25%	125.25MB	31.40MB		

表 7.1.8 模型压缩实际技术方案表

模型	原模型	稀疏度	剪枝后	量化后	总模型大小	FPS
CWM	112MB	40%	40.97MB	10.85MB	34.95MB	6.716
IGM	167MB	25%	94.39MB	24.10MB		

7.2 Correlation Module 加速优化

对于设计的深度学习模型，为了能够满足赛题方要求，必须对模型推理性能进行改进与优化。我们通过分析模型推理阶段的各模块对 CPU 资源占比与计算性能耗费占比，发现 Correlation Module 在计算衣服外观流特征和人体姿势外观六特征时对计算和存储资源的成本最高，因此我们注重采用新的高效算法优化该模块的计算。

7.2.1 算法调研分析

通过算法调研分析得到，Img2Col 算法可以用来高效处理矩阵的运算。该算法常用在卷积运算中，因为对矩阵进行卷积时，需要不断从矩阵中按特定步长截出一定大小的卷积区域。使用传统的 For 循环进行卷积操作，非常费时，遇到数据量大时，效率十分低下。

而 Img2Col 算法的核心就在于将卷积操作转化为矩阵操作，可以明显加快卷积速度。具体原理是将每一个输入特征子矩阵展开成一行（一列），生成新的输入特征矩阵，其行数和输入特征子矩阵的个数相同。同时将卷积核中的权重矩阵展开成一列（一行），多个权重矩阵可以排列成多列，最后将生成的输入特征矩阵与权重矩阵相乘即可得到相应结果。

Img2Col 算法充分利用了计算时进行访存的相邻性，将需要计算的特征子矩阵存放在连续的内存中，便于一次性地将所需要计算的数据直接按照需要的格式取出进行计算，这样便减少了内存访问的次数，从而减小了计算的整体时间。

表 7.2.1 Img2Col 算法基线性能测试

Correlation Accelerating			
Algorithms	CorrTorch	Img2Col	Acceleration Rate
$n = 1000$	4.6311	2.7948	1.6570
$n = 10000$	48.8326	30.0800	1.6234
Average time	0.004883	0.003008	1.6234

7.2.2 Img2Col 算法优化过程

通过对比分析 Correlation Module 计算与卷积计算的过程，利用 Img2Col 算法原理进行模块计算优化。

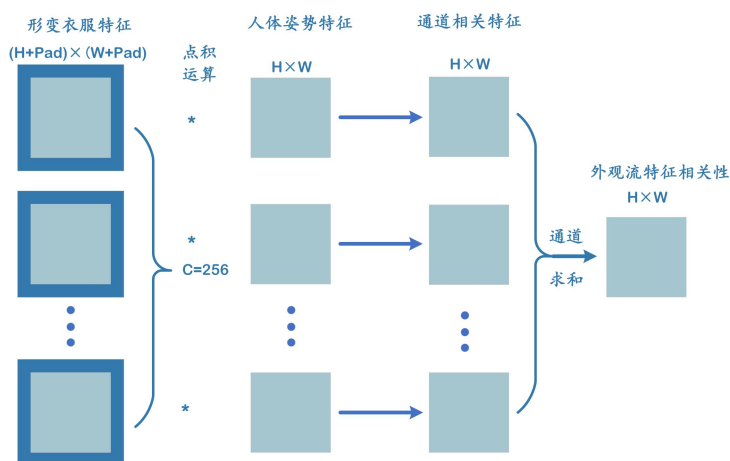


图 7.2.1 Correlation Module 计算原理

应用 Img2Col 算法对计算过程进行优化，具体优化原理如图 7.2.2 所示：

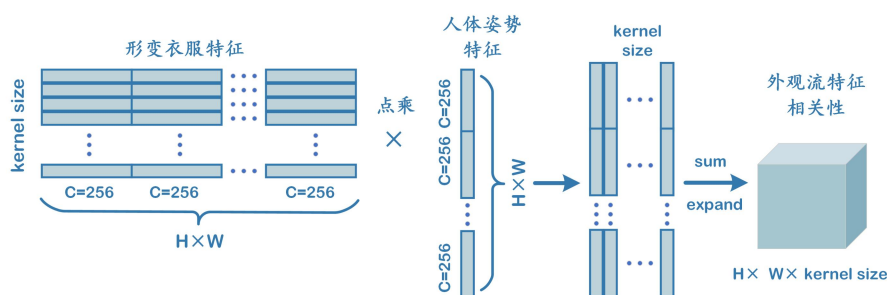


图 7.2.2 Img2Col 算法应用原理

对计算模块的重构调整，使得 Img2Col 算法成功无差错性地应用于模型的推理，进一步实现模型推理性能的优化，如表 7.2.2 所示：

表 7.2.2 Img2Col 算法性能对比

Model Accelerating				
Algorithms	CorrTorch	Img2Col	FPS	Acceleration Rate
$n = 1000$	147.8491	94.7902	10.81	1.5598
$n = 10000$	1489.1325	927.4293	10.77	1.6057
Average time	0.1488	0.0929	10.79	1.6017

7.3 OpenVINO 推理框架的应用

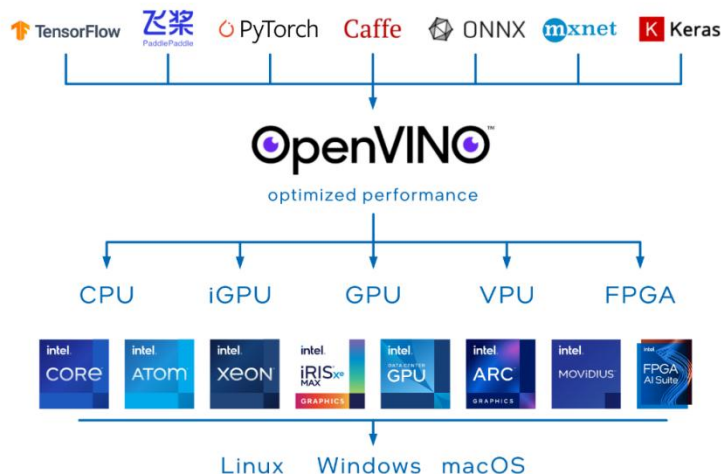
7.3.1 技术选型分析

推理框架是一种加载、优化和执行机器学习、深度学习模型的工具，专门用于模型推理(inference)，即使用已经训练好的模型进行预测或者推断。使用推理框架能够实现高效、低资源消耗的模型推理计算，使模型部署和运行变得高效。

目前主流的推理框架包括 TensorRT, ONNXRuntime, OpenVINO, MNN 等，在推理框架的选择上，我们主要考虑了以下几个方面：

- 性能角度：万兴科技在布置赛题时对模型最终的全流程推理速度，FPS 有要求，提升模型的性能非常重要
- 应用场景：最终的 EXE 可执行文件只能在 CPU 上运行，不能调用任何 GPU 资源，且测试场景为 Windows 平台，使用的 CPU 型号为 i7-8700/i7-10700
- 模型文件大小：考虑到最终需要上线部署，最终的 EXE 可执行需要体积需要尽可能小，保证轻量化

OpenVINO(Open Visual Inference and Neural Network Optimization)[10]是英特尔公司推出的一个全面的工具套件，用于快速部署应用和解决方案，它是一个 Pipeline 工具集，包括“Model Optimizer”、“Inference Engine”、“Model Zoo”等多个组件用于加速和优化深度学习推理任务的开发部署，同时可以兼容各种开源框架训练好的模型。



OpenVINO 针对 Inter 的 CPU 做了大量的优化，在针对赛题要求进行分析后，我们确定选用 OpenVINO 推理框架来对模型进行推理优化与部署。

7.3.2 模型转换流程

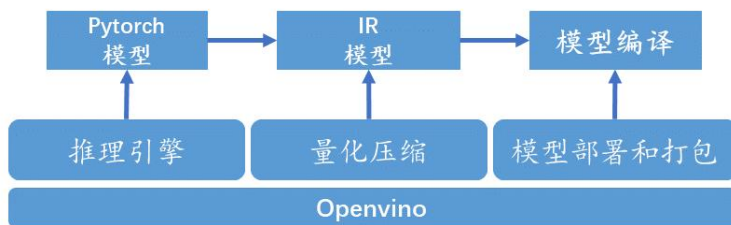


图 7.3.1 模型转换流程图

基于 PyTorch 的模型不能直接用 OpenVINO 进行集成和推理, 我们需要首先将 PyTorch 模型导出为 ONNX(Open Neural Network Exchange) 格式, ONNX[11]是由 Microsoft 和 Facebook 等公司共同开发和维护的开放式标准, 可以实现不同框架之间的无缝迁移。在将模型导出为 ONNX 格式后, 可以使用 OpenVINO 的“Model Optimizer”工具将 ONNX 优化为适用于特定硬件的中间表示 IR(Intermediate Representation)。Model Optimizer 提供了多种优化技术以提高推理性能和减少资源消耗, 这些优化选项只需要我们制定相应的参数即可, Model Optimizer 会自动帮我们执行优化, 在具体实践中, 我们采用了以下几种优化方法:

- 算子融合: 将卷积、批归一化和激活函数合并为一个操作, 减少内存传输和计算的次数
- 异步执行和并行处理: Model Optimizer 根据模型结构自动将模型中的操作划分为多个并行执行的部分, 以充分利用硬件的并行计算能力, 提高推理速度
- 图优化: Model Optimizer 对模型的计算图进行分析和优化, 去除不必要的计算和数据传输, 以减少推理过程的开销

在将 PyTorch 转换为 OpenVINO 可以支持的 IR 优化格式后, 接下来我们进行模型的推理。其中 IR 格式包含两个文件:

- 📁 .xml 文件: 存放模型的网络结构
- 📁 .bin 文件: 存放模型的权重

7.3.3 OpenVINO 推理性能优化

如果使用 Python 语言进行推理, 最终要生成一个 EXE 可执行文件, 只能选用 Python 的打包工具, 如 PyInstaller, Cx_Freeze 等, 但是需要将项目需要的依赖环境一并打包进去, 通常最终的可执行文件体积较大, 且模型加载速度也比较慢, 因此, 我们决定选用 C++来对模型进行推理, C++进行推理可以提供更高的性能、更低的资源消耗以及更加轻量化的可执行文件, OpenVINO 也提供了通用的 C++API。

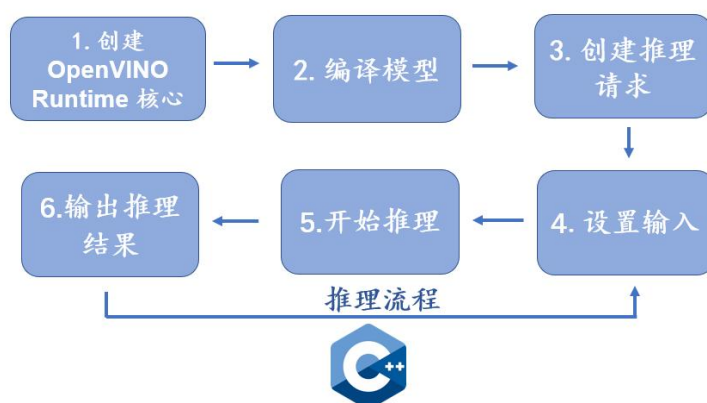


图 7.3.2 使用 Open VINO Runtime API 推理的流程

OpenVINO Runtime(运行时)是 OpenVINO 工具套件的一部分，提供了执行推理任务的运行时引擎，使用 OpenVINO Runtime 对执行推理的流程大致可以分为上述图中的六个方面：

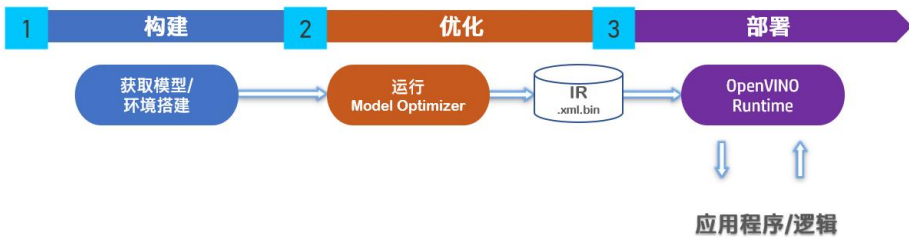
- 1) 创建 OpenVINO Runtime 核心(Inference Engine Core): 用于加载、配置和执行模型的推理任务；
- 2) 编译模型: 将模型转化为特点硬件上可执行代码，在编译过程中，Model Optimizer 会应用一系列优化技术提高推理性能；
- 3) 创建推理请求: 推理请求(Inference Request)用于将输入数据传递给模型并获取输出结果，创建推理请求又包含以下两个步骤：
 - 获取可执行网络(Executable Network)对象：使用 Inference Engine Core 加载 IR 模型并获取可执行网络对象；
 - 准备输入数据和接受数据的容器对象
- 4) 设置输入: 将 PyTorch 张量转换为 OpenVINO 支持的张量数据格式
- 5) 开始推理: OpenVINO Runtime 支持同步和异步两种模式的推理：
 - 🚦 同步推理(Synchronous Inference): 在执行推理请求时要等待推理任务完成并返回结果才可以执行后续操作；
 - 🚦 异步推理(Asynchronous Inference): 可以提高应用程序的整体帧速率，不需要等待推理过程的完成；

由上述分析我们决定选用异步推理的方式，以尽可能提高最终可执行文件的性能。

- 6) 处理推理结果；

在模型可以成功推理后，我们使用了上述说的几种优化方法来提高最终模型的性能。

7.4 项目部署



在软件工程中，部署是指将开发完毕的软件投入使用的过程。类似的，对于深度学习模型来说，模型部署是指让训练好的模型在特定环境中运行的过程。本赛题最终要求将模型部署成一个不需要额外安装任何依赖环境的.EXE 可执行文件，且对模型的推理速度、吞吐量和时延有要求。

为了满足高性能和跨平台部署的需求，我们选择使用 C++ 语言对模型进行部署。

7.4.1 部署准备

PyTorch 模型在使用 C++ 进行推理时，需要使用 OpenVINO 的 C++ 的 API 来处理，所以我们需要对 PyTorch 模型的使用的算子和数据类型进行替换，并采用 C++ 版本的 OpenCV(Open Source Computer Vision Library)视觉库进行图像处理的一系列操作。

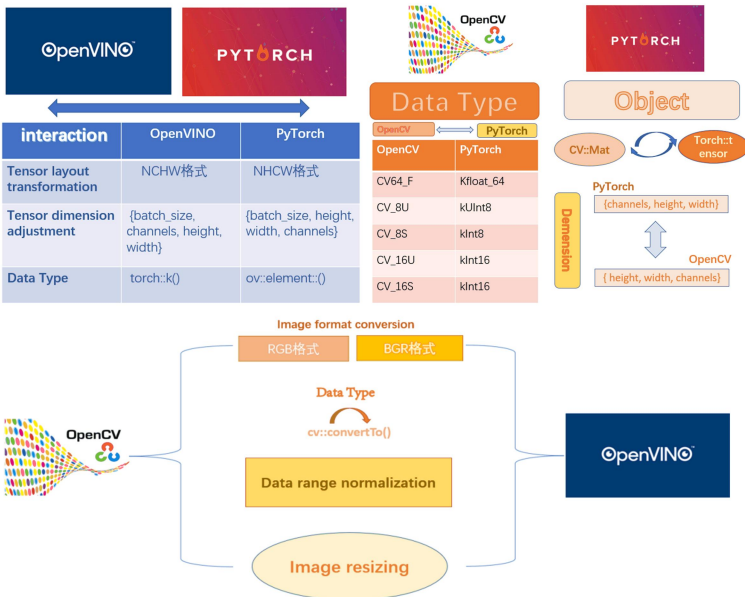


图 7.4.1 OpenVINO、PyTorch 与 OpenCV 三者数据交互转换

有上图可以看出三者之间进行数据交互和传递时需要进行转换，例如：

1. 张量布局：OpenVINO 只接受 NCHW 格式，且在将 PyTorch 的 Tensor 对象转换为 OpenVINO 的 Tensor 对象时，OpenVINO 要求分配的 Tensor 对象内存必须是连续的，这可以保证其在不同硬件平台上高效地进行处理和传输数据，因此需要使用 “.contiguous()” 方法确保 Tensor 对象内存是连续的。
2. 图像格式的转换：OpenCV 通常使用 BGR 格式来表示图像数据，其中颜色通道的顺序为 Blue-Green-Red，而 OpenVINO 使用 RGB 格式来表示，颜色通道的顺序为 Red-Green-Blue；
3. 数据类型的转换：OpenCV 使用不同的数据类型来表示图像数据，例如 ‘CV_8UC3’ 表示 8 位无符号整数的 3 通道图像；OpenVINO 使用 `ov::runtime::Tensor` 表示张量数据，其中数据类型可以是 `ov::element::Type` 中定义的数据类型；

7.4.2 .EXE 编译部署

在完成 C++ 的推理后，下一步是选择配置构建系统，编写相应的配置文件，根据选择的构建系统，执行构建命令来编译和链接项目，最终生成一个可执行文件。

我们团队主要使用了 Visual Studio 平台来进行 C++ 的项目构建与配置，使用 CMake(Cross-Platform Make)来管理和控制项目的构建过程，Cmake 提供了一种简洁的语法来描述项目的构建配置，我们可以通过编写 CmakeList.txt 来指定项目的源代码、依赖项、编译选项等从而简化构建过程。详细的 CmakeList 文件将在附录中给出。

最后，我们构建了一个 build.sh 脚本在命令行中编译推理代码，将编译过程自动化，最终生成需要的可执行文件，并存放到制定文件夹当中以方便使用。

7.4.3 部署中遇到的问题与挑战

在部署的过程中，我们团队遇到了一些不易发现的问题和难以解决的技术性挑战，下面将列出一部分问题并给出解决方案

- 将 PyTorch 模型导出为 ONNX 模型时算子不兼容：在模型的转换后对 ONNX 模型进行编译测试的时候出现报错，通过观察导出的 warning 后发现模型中使用了 `grid_sampler` 操作，这个算子的作用是通过采样网络对输入图像进行插值从而实现图像的几何变换，而这个操作在 ONNX 中一直没有被支持(最新的 ONNX opset 版本为 Version18),解决方法是使用 MMCV 库中的“`grid_sampler`”替换这个算子，具体的替换代码如下：

```
from mmcv.ops.point_sample import bilinear_grid_sample
import torch.nn.functional as F
img = bilinear_grid_sample(tenInput, grid, align_corners=False)
img_o=F.grid_sample(input=tenInput,
                     grid=g,
                     mode='bilinear',
                     padding_mode='border',
                     align_corners=True)
```

- 算子匹配问题：在将 PyTorch 的 Tensor 对象转换为 OpenVINO 的 Tensor 对象时，OpenVINO 要求分配的 Tensor 对象内存必须是连续的，这可以保证其在不同硬件平台上高效地进行处理和传输数据，因此需要使用 “.contiguous()” 方法确保 Tensor 对象内存是连续的；
- 像素值范围：浮点类型的张量如果直接用于图像显示，像素值通常会被解释为灰度值或者颜色通道值，这是如果像素值超过了可视化所需的范围，会导致图像显示为全白色，因此需要浮点类型的张量转换为整数类型，并将像素值映射到正确的范围内；
- CMake 默认的 build 模式为 Debug，在没有安装 Visual Studio 的 Windows 上运行会提示缺失 MSVCP140D.dll 等 dll 文件，需要使用 cmake --build . --config Release 来用 Release 模式构建；
- Cmake 用 Release 模式时，libtorch 需要使用 Release 的包，不能用 Debug 的包；

上述是一些我们在进行 C++部署时遇到的问题，最终通过查阅大量资料以及细致的 Debug，我们解决了遇到的问题与挑战，并得到了赛题所需的 EXE 可执行文件，并对 EXE 可执行文件进行了可移植性测试，验证了 EXE 可执行文件的实用性和可移植性。

7.4.4 本地 Web 部署

为方便广大消费者用户的使用，提供更加便捷的虚拟试衣体验，我们团队在企业方要求的制作本地 EXE 可执行程序的基础上，将模型部署到了本地的 Web 应用。用户可以直接通过浏览器访问我们的产品，无需下载安装额外的软件或进行相关的环境配置，在 Web 部署的过程中，我们从现实场景着想，确保 Web 应用的用户界面简洁美观，响应速度快，提供流畅的用户体验，并采取安全措施对用户的个人数据和隐私信息进行保护。下面将详细阐述 Web 部署的流程与配置。

- 前端部署

- Bootstrap：基于 HTML，CSS，JavaScript 的前端框架，提供了丰富的预定义组件和样式，能够快速搭建简洁美观的用户界面，使

得前端交互开发更加高效

- **jQuery**：用于简化 Javascript 编程，处理 DOM 操作、事件和动画效果。

我们使用了 **Bootstrap** 框架以及其他自定义的样式构建了两个前端界面，分别是用户登录界面以及虚拟试衣功能主界面，布局美观，交互性良好。

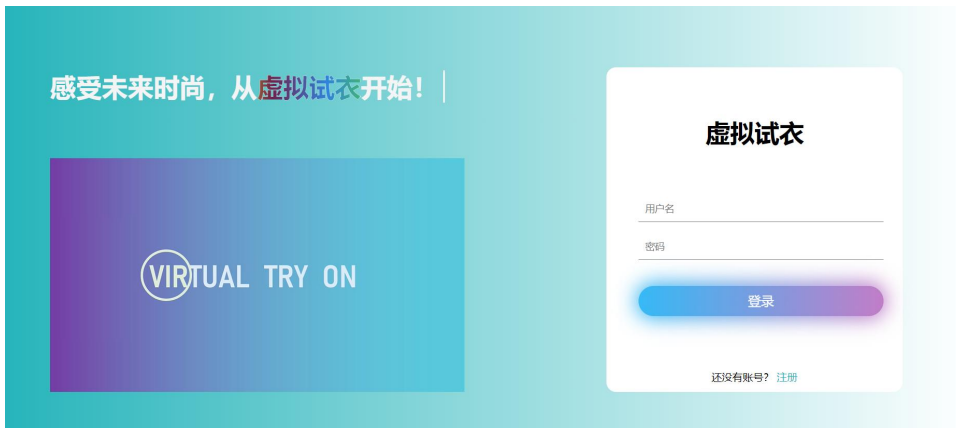


图 7.4.2 Web 登录界面

登录页面的左边是项目的宣传视频，右边设置了一个登录表单，包含登录和注册按钮，用户第一次使用可以首先注册一个账号，我们也提供了一个默认账号以供试用：

用户名	未名湖
密码	123456

在输入正确的账号和密码点击登录按钮后，将跳转至换衣的主界面。



图 7.4.3 Web 端主界面

虚拟试衣的功能界面包含一个顶部导航栏，左右各一个的图片容器以及具体的操作按钮等。导航栏具有提供产品具体信息，返回登录首页，查看此前换衣生成结果的功能。

功能描述：

- 1) 用户可以通过点击下方的操作按钮“选择人物图片”和“选择衣服图片”上传自己的图像和想要试衣的图像，也可以通过点击左右两侧的图片容器快速选择进行换衣。

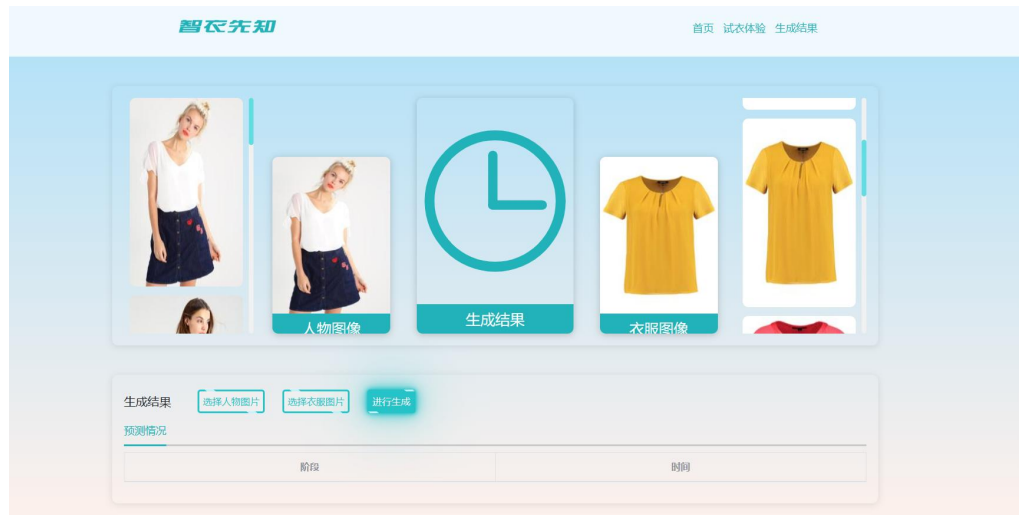


图 7.4.4 Web 上传图像界面

- 2) 在上传完图像后，用户只需要点击“进行生成”按钮即可实现实时换衣，我们在生成换衣图像的过程中使用“processTimer()”函数设置了一个进度条模态框，以便随时查看进度：

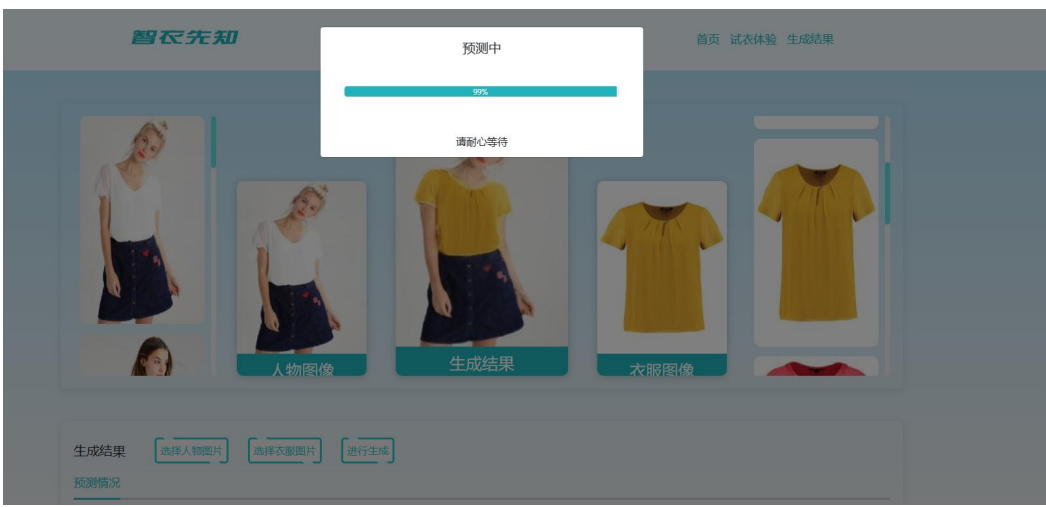


图 7.4.5 Web 虚拟试衣界面

- 3) 为了了解试衣生成的整体耗时情况，我们在下方设置了预测时间的表单，将反馈此次换衣所使用的前处理时间、推理时间和后处理时间。同时我们也考虑了异常情况，当用户上传的图像错误或者有其他故障，会显示“预测失败”的提示，并告知用户如何正确操作。

注：由于比赛要求产品不能够使用网络，因此我们采取了本地部署的方式，故省略了 Web 服务器部署以及 HTTP 配置等流程。

● 后端部署

Flask 框架轻量简洁、灵活可扩展，适用于小型和大型项目，与 Python 生态系统无缝集成，因此我们采用改框架完成 Web 后端的部署。

在后端代码中，我们定义了几个接口来实现虚拟试衣功能：

- ✓ '@app.route('/')': 定义了主页路由，该路由用于渲染前端的主页 'index.html'
- ✓ '@app.route('/favicon.ico')': 定义了获取网站图标的路由
- ✓ '@app.route('/predict', methods=['POST'])': 定义了预测路由，该路由接收前端发送的图片数据，进行后端推理，然后将预测结果返回给前端。在正确配置环境后，我们启动后端应用，并通过浏览器访问 'http://localhost:5000' 即可访问前端页面，进行虚拟试衣。

最后为了确保 Web 应用的安全性，我们加入了种种安全措施如输入验证、访问控制、强化密码策略、安全审计和日志记录等。

8. 算法评价

8.1 定量比较

下表列出了 CP-VTON、ClothFlow、CP-VTON+、ACGPN、WUTON 和我们提出的模型在 VITON 和 MPV 数据集上的 FID 分数。与 VITON 数据集上的基于解析器的方法相比，我们的 PF-AFN 在生成高质量试穿图像方面具有明显优势，大幅领先。在 MPV 数据集上，我们的模型也优于基于解析器的 WUTON，这证明了“教师-导师-学生”知识蒸馏方案的优越性。

表 8.1.1 不同算法 FID 比较结果

算法模型	FID
PF-AFN	9.92
SDAFN	11.8
GP-VTON	12.26
FS-VTON	13.87
HR-VTON	16.12
Ours	8.906

8.2 定性比较



图 8.2.1 定性分析效果示意图

我们主要将我们的方法与最近提出的基于解析器的方法进行视觉比较，包括 CP-VTON、ClothFlow、CP-VTON+和 ACGPN。如图所示，当参考人以复杂的姿势站立，例如双手叉腰或双手挡在身体前面时，目标服装与人之间会产生大的错位。在这种情况下，基准模型都无法将长袖衬衫映射到相应的身体区域，导致袖子断裂、袖子没有附着在手臂上和刺绣失真。实际上，这些方法无法模拟高度非刚性变形，因为它们的方法存在局限性，即 TPS 的自由度受限。在第二行和第三行中，基准方法生成的图像存在明显的伪影，例如混乱的下身服装和上身服装被映射到下身区域。这些基于解析器的模型对分割错误非常敏感，因为它们严重依赖于解析结果来驱动图像生成。我们的模型生成高度逼真的试穿结果，同时处理了衣服和人之间的大错位，保留了目标服装和非目标服装的特征，并保留了清晰的身体部位。除了以上优点外，由于在外观流上采用了二阶平滑约束，模型能够模拟服装和人之间的长程对应关系，避免了标志和刺绣上的扭曲。由于在训练过程中我们不对输入人物图像的任何信息（如服装或身体部位）进行掩码处理，模型能够自适应地保留或生成身体部位，从而保留身体细节。

将我们应用的方法与无解析器方法的先驱 WUTON 进行比较。WUTON 生成的结果存在明显的伪影，例如它无法区分上衣和下衣之间的边界，使得目标上衣被扭曲到了下半身区域。由于 WUTON 是由解析器方法生成的虚假图像进行监督的，因此很可能会导致结果不令人满意。相比之下，我们的模型能够精确地将衣服变形到目标人物上，即使在出现复杂的姿势时也能生成高质量的图像，同时保留目标衣服和非目标衣服（如裙子）的特征。模型受益于由真实图像进行监督以及通过提取外观流找到衣服和人之间准确的密集对应关系。

9. 算法边界测试

为了增强算法模型的鲁棒性，并且验证算法在输入边界值附近的行为和处理能力，需要针对一些特定的应用情况进行模型测试，确定算法边界。下面我们对模型进行了大量的测试，并根据测试结果作了简略的分析。

9.1 人物非常态姿势

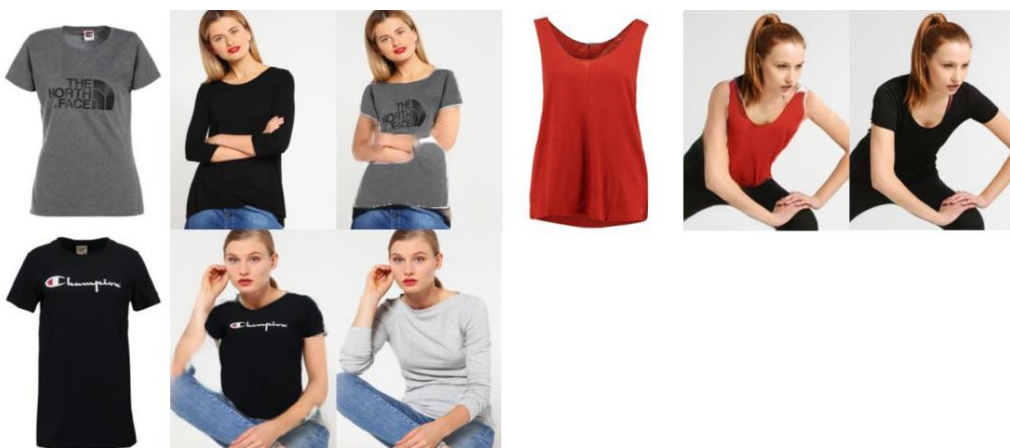


图 9.1 非常态姿势人物换衣效果图

可以看到，我们的模型对非常态姿势的人物也具有较好地适应性，对于姿势幅度不是很大的人物图像，试衣效果完全可以达到较好标准。不过对于较难分割的细节部分，会影响到试衣后衣服的顺滑自然效果，从这一点考虑我们还需要对算法的性能进行后续的优化。

9.2 特殊人群换衣

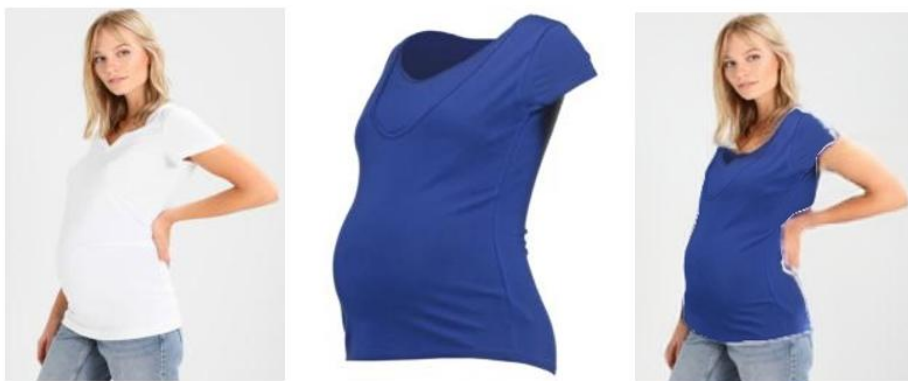


图 9.2 特殊人群换衣效果图(孕妇)

通过上图可以看出我们的模型对特殊人群比如孕妇，具备较好的适应性，对特定需求的人群提供适应的衣服仍能够保持较好的兼容性，这个特点仍可以拓展到其他类型群体，比如残疾人群体。

9.3 特殊款式衣服

从图 9.3 可以看出我们的模型对花纹复杂的衣服也具备极好的适应性，不会导致衣服的花纹款式发生显著变化。



图 9.3 有花纹的衣服换衣效果图

9.4 人物与衣服尺寸不匹配



图 9.4 人物与衣服尺寸不匹配换衣效果图

通过对人物与衣服尺寸不匹配的样例进行测试，我们发现，算法模型最终的效果不佳，不能将衣服的尺寸很好的与人物的身材相匹配。

9.5 不同衣服款式

可以看到，在对不同衣服款式进行换衣时，我们模型的最终换衣效果比较优秀，还可以预测出人物的手臂图像等。



图 9.5 不同衣服款式换衣效果图

10.项目总结和发展前景

我们的产品“智衣先知”基于深度学习算法，很好地实现了万兴科技对于虚拟试衣算法的几点要求，无论是模型大小，效果性能指标 FID，还是运行性能 FPS 均达到了不错的水准。创新性地使用了自适应的知识蒸馏，最后的模型既不依赖人体解析结果，同时又能对基于解析器结果生成图像做一定的监督调整，从而提高虚拟试穿的效果。同时，使用二阶平滑约束和可调节的知识蒸馏，使最终生成的人物试穿图像效果逼真可靠；我们还对模型进行了多重优化，提高了模型的训练和推理速度，最后，除了企业提供的测试集，我们还在其他虚拟试衣经典的数据集如 VITON, DeepFashion 对我们的模型进行了测试，均取得了不错的结果。

我们的项目“智衣先知”可以很好地解决在线购物中消费者难以准确判断衣服合适度和上身效果的问题。该项目支持消费者上传自己和服装的图像来实现线上的“试穿”，以便消费者更加真实地感受服装效果。与现有的其他虚拟试衣算法相比，我们的产品试衣效果更为逼真贴近，且响应速度快，在电商平台兴起的时代，具有极为广阔的市场前景和巨大的发展潜力。

参考文献

- [1] 曲欣,刘春艳,齐贝宁,刘莹,赵思雪,秦雅洁.AR 虚拟试衣技术发展现状及推广前景展望[J].内江科技,2023,44(01):87-88.
- [2] Bochao Wang, Huabin Zheng, Xiaodan Liang, Yimin Chen, Liang Lin, and Meng Yang. Toward characteristicpreserving image-based virtual try-on network. In ECCV, 2018.
- [3] Xintong Han, Xiaojun Hu, Weilin Huang, and Matthew R Scott. Clothflow: A flow-based model for clothed person generation. In ICCV, 2019.
- [4] Matiur Rahman Minar, Thai Thanh Tuan, Heejune Ahn, Paul Rosin, and Yu-Kun Lai. Cp-vton+: Clothing shape and texture preserving image-based virtual try-on. In CVPR Workshops, June 2020.
- [5] Han Yang, Ruimao Zhang, Xiaobao Guo, Wei Liu, Wangmeng Zuo, and Ping Luo. Towards photo-realistic virtual try-on by adaptively generating-preserving image content. In CVPR, 2020.
- [6] Thibaut Issenhuth, J. Mary, and Clement Calauz ´ enes. Do ` not mask what you do not need to mask: a parser-free virtual try-on. In ECCV, 2020.
- [7] Lin, T.-Y., Dollár, P., Girshick, R., He, K., Hariharan, B., & Belongie, S. (2016). Feature Pyramid Networks for Object Detection. arXiv e-prints.
- [8] Sun, K., Xiao, B., Liu, D., & Wang, J. (2019). Deep High-Resolution Representation Learning for Human Pose Estimation.
- [9] Cao, Z., Simon, T., Wei, S.-E., & Sheikh, Y. (2017). Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields.
- [10] <https://www.intel.cn/content/www/cn/zh/developer/tools/opencv-toolkit/overview.html>
- [11] <https://onnx.ai/>
- [12] <https://paperswithcode.com/dataset/viton>
- [13] <https://hyper.ai/datasets/20586>
- [14] <https://mvp-dataset.github.io/index.html>

附录

开发平台			
PyCharm	2022.3.2	Visual Studio Code	1.80.1
Visual Studio	17.5.5		
开发环境			
neural-compressor	2.2.1	nncf	2.5.0
numpy	1.23.4	onnx	1.14.0
opencv-python	4.7.0.72	onnxruntime	1.15.1
openvino	2022.3.0	pandas	1.3.5
pytorch-fid	0.3.0	rembg	2.0.50
pytorch	2.0.0	torch-pruning	1.1.9
intel-openmp	2021.4.0		

CMakeList 配置文件

```
cmake_minimum_required(VERSION 3.10)
project(tryon)

list(APPEND CMAKE_PREFIX_PATH "../libtorch_release/libtorch/share/cmake")
list(APPEND CMAKE_PREFIX_PATH "../opencv/build")
list(APPEND CMAKE_PREFIX_PATH "../openvino/runtime/cmake")

add_executable(try_on main.cpp)
find_package(OpenVINO REQUIRED COMPONENTS Runtime)
find_package(OpenCV REQUIRED )
find_package(Torch REQUIRED)

include_directories( ${OpenCV_INCLUDE_DIRS} )
set(ov_link_libraries openvino::runtime)

#target_include_directories(try_on PRIVATE "../common")
target_link_libraries(try_on PRIVATE ${ov_link_libraries} ${OpenCV_LIBS}
${TORCH_LIBRARIES})

if (MSVC)
    file(GLOB TORCH_DLLS "${TORCH_INSTALL_PREFIX}/lib/*.dll")
    add_custom_command(TARGET try_on
        POST_BUILD
        COMMAND ${CMAKE_COMMAND} -E copy_if_different
        ${TORCH_DLLS}
        $<TARGET_FILE_DIR:try_on>)
endif (MSVC)
```



中国大学生服务外包创新创业大赛